

COMP101

Introduction to Programming

2025-26

Lecture-16

Iteration while – Python

while loop

- ‘non-deterministic loop’
 - we don’t know when the loop is to stop
- Loop is controlled with a test condition at start
 - Test condition is same as selection – True/False
- Start the loop
 - while condition = True iterate the loop body
- Stop the loop
 - when condition = False exit the loop
 - Must change the test condition to False
 - In the loop body to make the loop stop

while loop

- `while` `CONDITION` :
 `<STATEMENTS>`

Three elements in loop header

`i = 0`

`while (i < 5):`

`i = i + 1`

`print (i)`

`print ("loop ended")`

'sentry variable' initialised as close as possible to loop:
Zero for numbers, space for string. Use it in test condition at start of loop

Must change value of sentry var in loop body to avoid an infinite loop.
i must become >5 eventually

First un-indented line that lines up with 'while' shows the loop has ended

while loop: error infinite loop

```
name = input ("Enter your name: ")
```

```
onScreen = input("Want to see your name on screen? Y/N: ") #sentry  
onScreen = onScreen.upper() #convert i/p to upper-case
```

```
while onScreen == "Y": #loop starts when i/p = 'Y'  
    print (name)  
    print() #blank line – aid readability  
    #a line is missing - need to input the sentry again or loop goes infinite  
print("Loop end")
```

Ctrl-C will stop an infinite loop

This will form an 'infinite loop' as the sentry var is not changed in the code. It will just keep outputting the name down a scrolling screen

while loop: infinite loop fixed

```
name = input ("Enter your name: ")
```

```
onScreen = input("See your name on screen? Y/N: ")      #sentry var  
onScreen = onScreen.upper()
```

```
while onScreen == "Y":      #when input='Y', loop starts: when input='N', do not start the loop  
    print (name)  
    onScreen = input ("See your name again? Y/N: ") #input Y to loop again or N to exit loop  
    onScreen = onScreen.upper()  
    print()
```

```
print("Loop end")      #loop ended - line executes when input is 'N'
```

input='Y' execute all indented code as a sequence – all lines execute.
Input='N' loop exits either by not having started or by sentinel being changed to 'N' in loop body
First un-indented line outside the loop always executes – loop ended

While loop: logic error1

Infinite loop Exit condition can't be met	Check the logic
In a game, Hero starts with 7 lives; If hurt in battle 3 lives are lost; Hero dies when <code>lives_left = 0</code> <pre>lives_left = 7 while lives_left != 0: print ("Hero is hurt in battle - lose 3 lives") lives_left = lives_left-3 print ("Hero has",lives_left,"lives left") print ("Hero has no lives remaining") print ("Loop end") #here, lives_left controls the loop - no user input; #in each iteration loop checks 'lives_left' to #determine whether to loop again or not</pre>	Subtracting 3 from 7 repeatedly will never give zero: $7 - 3 = 4$ $4 - 3 = 1$ $1 - 3 = -2$... The number just keeps getting bigger (negatively) Check the initialisation of <code>lives_left = 7</code> Make it a multiple of 3 Or change test to <code>>0</code>

while loop: logic error2

**Unexpected output:
Loop body works for i=0
but o/p is wrong**

#expected output is 1 to 5 inclusive
#i=0 is not best solution

i = 0

while i <= 5:

 i = i + 1
 print (i)

1
2
3
4
5
6

What's this doing here?

Dry run

i	i <= 5 True?	i=i+1	print
0	Y	1	1
1	Y	2	2
2	Y	3	3
3	Y	4	4
4	Y	5	5
5	Y	6	6
6	N	exit	exit

while loop: logic error2 fixed

Correct o/p:
Initialise to i=1 and
Swap body sequence

#actual output is now 1 to 5
#if we initialize i=0 then 0 will
o/p, so initialize as i=1

```
i = 1
```

```
while i <= 5:
```

```
    print(i)  
    i = i + 1
```

1
2
3
4
5

Dry run

i	i <= 5 True?	print	i = i+1
1	Y	1	2
2	Y	2	3
3	Y	3	4
4	Y	4	5
5	Y	5	6
6	N	exit	exit

while loop: logic error2 another fix

**Initialise to 1 and
change test to $i < 5$**

#actual output is now 1 to 5
inclusive

`i = 1`

`while i < 5:`

`i = i + 1`
 `print(i)`

1
2
3
4
5

Dry run

i	i < 5 True?	i = i+1	print
0	Y	1	1
1	Y	2	2
2	Y	3	3
3	Y	4	4
4	Y	5	5
5	N	exit	exit

while loop: construction

- Requirement:
 - Access a program via a main menu
 - single-entry via main menu
 - single-exit via main menu

Main Menu

A: Option-A

B: Option-B

C: Option-C

X: Exit the program

Menu: first thing seen . . . and last thing seen on exit;
When A or B or C is selected, output to confirm option;
When output is read, return user to menu
Exit from program is only via option X

while loop: construction step-1

- Step-1: Output a menu – sequence construct

```
print ("Main Menu")
print ("A: Option-A code")
print ("B: Option-B code")
print ("C: Option-C code")
print ("X: Exit")
print()                                #o/p a blank line for readability
option = input("Enter an option A,B,C or X to exit: ")
option = option.upper()                #user i/p converted to uppercase – makes tests easier
```

Analysis:

Program ends when valid option is input – but at least it shows the program is working with a menu.

It also crashes on invalid input.

What we need is to show a message to user what option they chose and then return to menu after reading the message

while loop construction step-2

- Step-2: Add the **selection** construct

```
print ("Main Menu")
print ("A: Option-A code")
print ("B: Option-B code")
print ("C: Option-C code")
print ("X: Exit")
print
option = input("Enter an option A,B,C or X to exit: ")
option = option.upper()
```

```
if option == "A":
    print ("Option-A under development")
elif option == "B":
    print ("Option-B under development")
elif option == "C":
    print ("Option_C under development")
else:
    print ("Program ended")
```

When A,B,C selected o/p a 'stub' to show option works;
Program ends after stub is output;
When X selected program ends with message 'Program ended';
When input is not A,B,C,X program ends with 'Program ended';

We need to return user to menu after a stub is o/p

while loop construction step-3

- Step-3: add the **iteration** construct

```
option = ' '
```

```
while option not in "X":
```

```
    print ("Main Menu")
```

```
    print ("A: Option-A code")
```

```
    print ("B: Option-B code")
```

```
    print ("C: Option-C code")
```

```
    print ("X: Exit")
```

```
    print
```

```
    option = input("Enter an option A,B,C or X to exit: ")
```

```
    option = option.upper()
```

```
if option == "A":
```

```
    print ("Option-A under development")
```

```
elif option == "B":
```

```
    print ("Option-B under development")
```

```
elif option == "C":
```

```
    print ("Option-C under development")
```

```
else:
```

```
    print ("Program ended")
```

#display menu unless X is entered

Note the selection is now indented under the while – it is part of the loop

A or B or C chosen – show stub;
User reads stub - return to menu;
X chosen - program ends;

Program ends on invalid input -
validation is needed

```
option = ' '
```

```
while option not in 'X':
```

```
    print ("Main Menu")
```

```
    print ("A: Option-A code")
```

```
    print ("B: Option-B code")
```

```
    print ("C: Option-C code")
```

```
    print ("X: Exit")
```

```
    print
```

```
    option = input("Enter an option A,B,C or X to exit: ")
```

```
    option = option.upper()
```

```
    while option not in('A','B','C','X'):
```

```
        print ('Invalid Option: ')
```

```
        option = input("Enter an option A,B,C or X to exit: ")
```

```
        option = option.upper()
```

```
#lines below execute on valid option: indentation is under inner loop
```

```
if option == "A":
```

```
    print ("Option-A under development")
```

```
elif option == "B":
```

```
    print ("Option-B under development")
```

```
elif option == "C":
```

```
    print ("Option_C under development")
```

```
else:
```

```
    print ("Option-X - loop ended")
```

Step-4: VALIDATION

This while loop is the 'outer loop'.
Only task is to output a menu

This while loop is the inner loop ('nested')
It is indented under the outer loop;
Checks for correct input.

Meets the specification;
User can only input A,B,C,X
User is returned to menu after A,B,C
User can end program with X

while loop: construction step-5

- Step-5: Cosmetics/Usability

Not critical code but makes the program more user-friendly, easier to use, easier to read etc

Make the actual code easier to read, easier to understand, easier to maintain c e.g.

When a stub is output, 'hold' it on screen so user can read it – use `input()`
Then ask user to press Return so they are taken back to the menu;

Use more 'whitespace' when outputting to make it easier to read – use `print()`

Use more 'whitespace' and essential commentary in the code itself - 'self documenting'

Improve messaging to the user when they input something in error

while loop testing

- Check loop body executes at least once
- Check the test condition will be met - to avoid an 'infinite loop'
- For unexpected output
 - Check the test condition
 - Check the order of statements in the loop body
 - Check the initialisation of the control var

while loop summary

- while (test = T):
 - execute the 'loop body' (indented code)
- Nested while - loop body can contain:
 - while-else
 - break
 - continue
 - another loop
 - selection
- Test must be input again as last line of the loop body to execute the loop again or to exit the loop
- No 'do' or 'endwhile' in Python